Computer Vision Term Project

1. Introduction

The term project is designed to provide students with hands-on experience in building, evaluating, and analyzing a complete Computer Vision system. Students will investigate both classical computer vision techniques and deep learning approaches to solve a real-world image classification problem.

The project emphasizes structured experimentation, critical analysis, reproducibility, and scientific writing. The final outcome must be presented as a research-style technical paper.

2. Problem Description

Students have the freedom to choose their own project idea, provided it meets the core technical requirements. The primary objective is to develop a robust image classification system capable of maintaining high performance under realistic visual challenges such as:

- Illumination variations
- Rotation and scaling
- Partial occlusion
- Synthetic camera distortion

The system must be evaluated not only on clean data but also under controlled distortion scenarios to assess robustness.

3. Dataset

Students must select a suitable dataset that aligns with their chosen problem.

Dataset Characteristics:

- Must be a multi-class classification task.
- Must contain enough samples to effectively train a Deep Learning model.
- Must be manageable for training on platforms like Google Colab.

Students may optionally introduce synthetic distortions to simulate real-world conditions.

4. Project Components

Each group must implement and evaluate the following:

4.1 Classical Computer Vision Baseline

The classical pipeline must include:

- Image preprocessing (color space selection must be justified)
- Edge detection (e.g., Canny)
- Optional shape detection (e.g., Hough Transform)
- Feature extraction (e.g., HOG, SIFT or ORB)
- Classifier (e.g., SVM)

The classical baseline should serve as a reference for comparison with deep learning approaches.

4.2 Convolutional Neural Network (CNN) Baseline

Students must:

- Design a custom CNN architecture (Choices must be clearly justified)
- Train the model from scratch
- Plot training and validation curves
- Analyze overfitting behavior

Clarification on "Custom Architecture" & "Training from Scratch": This does **not** mean writing the low-level mathematical operations of the network layers. It means you must **invent your own unique network architecture** by stacking layers (using frameworks like TensorFlow/PyTorch) and train it with random initial weights. For this baseline, **you are strictly not allowed** to use pre-existing standard architectures (e.g., VGG, ResNet, MobileNet) or apply Transfer Learning (pre-trained weights).

4.3 Improved Model and Experimental Study

Students must investigate at least one of the following:

- Data augmentation impact study
- Transfer learning using a pretrained model
- Color space comparison (RGB vs HSV vs grayscale)
- Robustness under synthetic distortions
- Lightweight model design for efficiency

A minimum of three controlled experiments must be conducted.

5. Experimental Protocol

Each group must:

- Split data into training, validation, and testing sets
- Use consistent evaluation metrics
- Clearly isolate variables during experiments
- Ensure reproducibility (random seed control)

Required evaluation metrics:

- Accuracy
- Precision and Recall
- Confusion Matrix
- Performance under distortion scenarios

6. Deliverables

6.1 Code Submission

- Google Colab notebook or GitHub repository
- Clean, well-documented code
- Clear execution instructions

6.2 Technical Paper (6–8 Pages)

The paper must follow this structure:

1. Abstract
2. Introduction
3. Related Work
4. Methodology
5. Experiments and Results
6. Discussion
7. Conclusion
8. References

Academic writing standards must be followed.

6.3 Presentation

- 30 minutes per group
- Clear explanation of methodology
- Visual presentation of results

---

7. Evaluation Criteria

- Problem formulation and clarity – 5%
- Classical baseline implementation – 20%
- CNN baseline implementation – 20%
- Experimental design and robustness study – 20%
- Analysis and discussion – 20%
- Code quality and reproducibility – 5%
- Paper quality and clarity – 10%

---

8. Optional Bonus: Advanced Capabilities

Groups may earn bonus marks (**up to +5% total**) by successfully implementing one of the following advanced tracks. This must be clearly documented in the technical paper under a dedicated subsection (e.g., "Deployment Strategy" or "Adversarial Robustness Analysis").

Option A: Deployment Strategies Take your trained model out of the notebook and deploy it for real-world interaction. Choose **one** of the following:

- **Live Stream (External Camera Deployment):** Connect the model to a dedicated external camera (e.g., an IP camera, a smartphone camera feed via a network, or a separately mounted USB camera) for real-time video frame processing and classification.
- **APIs:** Use FastAPI or Flask to create a backend API that accepts an image via POST request and returns classification results in JSON format.
- **Hugging Face Spaces:** Deploy a web application (using Streamlit or Gradio) to a public URL where anyone can interactively test the model.
- **Docker Containerization:** Create a `Dockerfile` that packages the model, API, and all dependencies for easy, environment-agnostic deployment.
- **Model Optimization:** Convert the trained model to TensorFlow Lite or ONNX formats, demonstrating its readiness for lightweight deployment on mobile devices or edge hardware (e.g., Raspberry Pi).

Option B: Adversarial Attacks Study Analyze the vulnerability of your deep learning model from a security perspective.

- Implement at least one adversarial attack method (e.g., Fast Gradient Sign Method - FGSM).
- Evaluate model performance under adversarial perturbations and visualize the adversarial examples.
- Provide a quantitative comparison between clean accuracy and adversarial accuracy, along with a brief discussion on potential defense strategies (e.g., adversarial training).
- This section must be clearly documented in the technical paper under a dedicated subsection titled "Adversarial Robustness Analysis."

---

9. Academic Integrity

- All external resources must be properly cited.
- Code reuse must be clearly referenced.
- Plagiarism will result in zero credit.

---

10. Expected Learning Outcomes

Upon successful completion of this project, students will be able to:

- Build and evaluate complete computer vision pipelines
- Compare classical and deep learning approaches
- Analyze robustness and generalization
- Conduct structured experiments
- Present results in a research-oriented format